GREECON
PLATFORM

Build the station from zero

Hands-on tutorial for the technician

From an empty desk to a soil moisture station that sends a reading to Greecon Platform every 10 minutes. Follow it in order. Every stage ends with a **check** that tells you it worked before you go on.

Companion to	*Soil moisture pilot: setup guide for the project manager* (what to do and when). This tutorial is the *how*.
Firmware	`devices/esp32-soil-moisture` in `greecontech/pilot`, compiled with `esp32` core 3.3.12
What is proven	The platform side was run end to end and the firmware compiles. **The firmware has not yet run on a real board: you are the first.** Write down everything unexpected.
Version	1.0 · 13 October 2026

Output samples marked “example” show the format, not your numbers. Values such as 2,950 or 1,320 are illustrations.

The road map

Eight stages, in this order. Stages 1 and 2 need no hardware, so do them while the parts are still in the post.

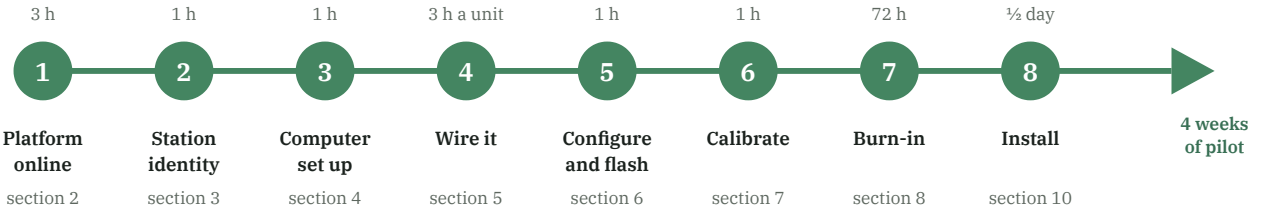


Figure 1. Hands-on time for each stage. The 72-hour burn-in is waiting time; you can build the second unit meanwhile.

1	**Before you start**	Parts, tools, labels	p. 3
2	**Put the platform online**	Railway, variables, first sign-in	p. 3
3	**Give the station an identity, and rehearse**	Site, source, key, simulated readings	p. 4
4	**Set up your computer**	Arduino IDE, ESP32 core, libraries	p. 5
5	**Wire the station**	Wiring cards, soldering, checks before power	p. 5
6	**Configure and flash**	config.h line by line, first upload	p. 6
7	**Calibrate the soil sensors**	Dry and wet values, a worked example	p. 7
8	**Burn-in**	72 hours, pass or fail, alert test	p. 7
9	**Power: mains and solar**	The reliable option, and the one that needs testing	p. 9
10	**Install at the farm**	Box, sensors, signal, hand-over	p. 9
11	**When it does not work**	Symptoms, messages, fixes	p. 10
A	**Reference**	config.h settings, what the station sends, commands	p. 11

HOW THE STATION WORKS, IN 30 SECONDS

The ESP32 sleeps. Every 10 minutes it wakes, switches on power to the sensors for a moment, reads two soil moisture sensors and a soil temperature probe, joins Wi-Fi, sends the readings to the platform over HTTPS with its key, and goes back to sleep. The platform stores them, draws them, checks them against the limits and emails the farmer. **If Wi-Fi is down at that moment, that reading is lost**: the station does not store readings. Signal strength is therefore what protects the 95% availability target.

1 · Before you start

Parts for one station

PART	QTY	TEST IT NOW
ESP32 DevKitC, classic ESP32-WROOM-32	1 (+1 spare)	Plug into a computer by USB. A serial port should appear (step 4.5). If the board is a bare module or has no USB socket, stop and ask Eridon.
DFRobot SEN0193 capacitive soil moisture sensor	2 (+1 spare)	Look for cracks or a missing component. The sensor's electronics are at the top; the **white line** marks how deep it may go.
DS18B20 waterproof temperature probe + **4.7 kΩ** resistor	1 (+1 spare)	Three wires: red, black, yellow. The resistor's value is printed in colour bands: yellow, violet, red.
IP65 box with cable glands	1	The lid seals; the glands fit the cables.
5 V USB adapter and a USB **data** cable	1	A charge-only cable lets the board power up but never shows a port.
Perfboard, jumper wires, heat-shrink, solder, sealant, cable ties	1 lot	–
SHT31 breakout (optional, greenhouse)	0–1	Only if the plot is a greenhouse.

Tools: soldering iron and solder, multimeter, small screwdrivers, wire cutters and strippers, a heat gun or lighter for heat-shrink, a glass of water, a laptop with a USB port.

Label everything

Write "Station 1" on the board and the box. Mark each sensor and its cable with a tag: **S1-A** for the shallow sensor, **S1-B** for the deep one. A sensor swapped by mistake gives a calibration that looks wrong and takes an hour to find.

> **KEEP A BUILD NOTEBOOK**
> Date, what you did, what you saw, anything odd. Since this firmware has never run on a board, your notes become part of the pilot's test results (deliverable D5).

2 · Put the platform online

The platform is the half of the system that already works. Skip this stage if Eridon confirms it is already deployed; you still need an administrator account.

1. **Open a Railway account** at railway.com with the company email. Choose the **Hobby** plan and turn on two-factor sign-in. (Railway's Hobby plan blocks outbound SMTP: that is why the platform sends email through Resend's web API.)
2. Choose *New Project → Deploy from GitHub repo* and select the Greecon Platform repository. Railway reads `railway.json` and builds the `Dockerfile`.
3. In the same project choose *+ New → Database → PostgreSQL*.
4. Open the app service, go to *Variables* and add these. Generate `SESSION_SECRET` with your password manager (40+ random characters) or with `openssl rand -base64 48`.

```
DATABASE_URL=${{Postgres.DATABASE_URL}}
SESSION_SECRET=<40 or more random characters>
ADMIN_EMAIL=<your company email>
ADMIN_PASSWORD=<a strong password, 8+ characters>
ADMIN_NAME=<your name>
SEED_DEMO=false
SIMULATOR=false
PROJECT_START_DATE=2026-08-01
APP_TIMEZONE=Europe/Tirane
```

The `ADMIN_*` variables only matter on the very first start, when the user table is empty. `SEED_DEMO=false` and `SIMULATOR=false` keep demo data out of the pilot. Email variables are added in the setup guide (Phase 2).

5. Open the service's *Settings* → *Networking* → *Generate Domain*. Wait for the deployment to turn green (a minute or two).
6. Open `https://<your-domain>/api/health` in a browser.
7. Open the domain itself, sign in with `ADMIN_EMAIL` and `ADMIN_PASSWORD`, go to *Account* and **change the password**.

> **CHECK**
> The health page shows `{"status":"ok","database":"ok","time":"…"}` and you can sign in. If it shows `"status":"degraded"`, the app cannot reach PostgreSQL: check that `DATABASE_URL` is exactly as above and that the database service is running.

3 · Give the station an identity, and rehearse

Every device has its own **key**. You create the site and the device on the platform first; the key is what the station presents when it uploads. You will first use a throw-away device called "Rehearsal", so test data never mixes with pilot data.

1. Go to *Sites & sources*. Under *Add a site*, enter the name "Pilot farm", sector **Agriculture**, and the location. Press *Add site*.
2. On the new site choose *Add a data source to this site*. Kind: **Device**. Name: **Rehearsal**. Press *Add source*.
3. **Copy the key now.** It looks like `gk_4c078cc4_…` and is shown only once. Put it in the password manager. If you lose it, use *New API key* on that source.
4. On a computer with Python 3, in the repository folder, send a simulated reading:

```
$ python3 devices/emulator/send_test_readings.py --url https://<your-domain> --key gk_xxxxxxxx_…
201 {"inserted":6,"alertsOpened":0,"metricsCreated":6}   (example: the format is real, taken from a test run)
```

5. Refresh *Sites & sources*. Six metrics have appeared under "Rehearsal": `soil_moisture_1`, `soil_moisture_2`, their two `_raw` twins, `soil_temp` and `wifi_rssi`. They are created automatically on the first upload.
6. Click *Edit* on **Soil moisture 1**, set **Lower limit 28**, press *Save*. Do the same for **Soil moisture 2**.
7. Simulate dry soil:

```
$ python3 devices/emulator/send_test_readings.py --url https://<your-domain> --key gk_xxxxxxxx_… --dry
201 {"inserted":6,"alertsOpened":2,"metricsCreated":0}
```

> **CHECK**
> Two alerts appear on the *Alerts* page and the dashboard. After the email service is set up (setup guide, step 2.8) one email arrives for both. A normal run of the emulator without `--dry` closes the alerts. A wrong key answers `401 {"error":"Invalid API key"}`; a deactivated source answers `403`.

4 · Set up your computer

1. Download and install the **Arduino IDE 2** from arduino.cc/en/software.
2. Open *File → Preferences* (*Arduino IDE → Settings* on a Mac). In **Additional boards manager URLs** paste exactly:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

3. Open the **Boards Manager** (left sidebar), search **esp32**, choose **"esp32 by Espressif Systems"**, pick version **3.3.12** in the version list and press *Install*. It downloads several hundred megabytes: use a good connection.
4. Open the **Library Manager** and install, each at the stated version: **OneWire 2.3.8**, **DallasTemperature 4.0.6**, **Adafruit SHT31 Library 2.2.2**. When asked about dependencies, press *Install all* (this adds Adafruit BusIO). These are the exact versions that compiled the firmware; newer ones probably work, but if anything breaks, go back to these.
5. Plug the ESP32 into the computer with the USB **data** cable. In *Tools → Port* a new port should appear (`COM3` or similar on Windows, `/dev/cu.usbserial…` on a Mac). If none appears: try another cable, then install the driver for the USB chip printed on the board (CP210x from Silicon Labs, or CH340 from WCH).
6. Open the sketch: *File → Open…* → `devices/esp32-soil-moisture/esp32-soil-moisture.ino`. Select *Tools → Board → esp32 →* ***ESP32 Dev Module*** and the port from step 5.
7. In that same folder copy `config.example.h` to `config.h` (the real file with secrets is never committed to git).
8. Press the tick button (**Verify**) to compile.

> **CHECK**
> The compile ends with a line like "Sketch uses 1062600 bytes (81%) of program storage space". The figures differ slightly by IDE version; red error text does not. An error "fatal error: config.h: No such file" means step 7 was skipped.

5 · Wire the station

Do this with **no power connected**. The board has pins named on its silkscreen (`GPIO25` may be printed as `25` or `D25`); pin positions differ between makers, so always go by the names.

switched 3.3 V power, from GPIO25 · **GND** ground · **signal** data back to the board · **3V3** constant 3.3 V

Soil sensor 1 (shallow)

SEN0193, three wires

VCC	→	GPIO25
GND	→	GND
AOUT	→	GPIO34

Soil sensor 2 (deep)

SEN0193, three wires

VCC	→	GPIO25
GND	→	GND
AOUT	→	GPIO35

Soil temperature probe

DS18B20 and a 4.7 kΩ resistor

red	→	GPIO25
black	→	GND
yellow (data)	→	GPIO4

4.7 kΩ resistor between GPIO4 and 3V3

Air sensor (greenhouse only)

SHT31 breakout. Leave out if not used

VIN	→	3V3
GND	→	GND
SDA	→	GPIO21
SCL	→	GPIO22

- **GPIO25 feeds three wires** (both soil sensors and the probe's red wire). The firmware switches it on only while measuring: that saves energy and slows corrosion of the sensors.
- **The soil sensors must stay on GPIO34 and GPIO35.** These are ADC1 pins; ADC2 pins stop working while Wi-Fi is on.

- Power to the board is separate: USB into the board's socket (section 9).

Build it

1. **Plan the layout** on paper first: the board on two rows of female headers soldered to the perfboard (so you can swap a dead board), and three rails: switched power (from GPIO25), ground, and 3V3.
2. **Solder the headers and the rails.** Keep the switched rail **separate** from 3V3. Joining them is the most common mistake: sensors would then be powered all the time.
3. **Connect each sensor through a connector** (a 3-pin header or a screw terminal) rather than soldering the sensor cable to the board, so a failed sensor can be swapped in the field in a minute. Check what the other end of the supplied sensor cable is: Dupont sockets plug into pin headers; bare wires need a terminal block or soldering.
4. **Fit the 4.7 kΩ resistor** between the probe's data line (GPIO4) and 3V3.
5. **Seal each soil sensor's electronics:** cover the top, where the cable joins the board, with heat-shrink or sealant. Never cover the lower, sensing part. The white line is the deepest the sensor may go into soil.

> **CHECKS BEFORE YOU APPLY POWER**
>
> - **No short:** multimeter on continuity (beep) between the board's `3V3` and `GND`: it must **not** beep. Do the same between the switched rail and GND, and between the switched rail and 3V3.
> - **Right pins:** with continuity, touch each sensor's AOUT lead and the matching GPIO pin: GPIO34 ↔ S1-A, GPIO35 ↔ S1-B, GPIO4 ↔ probe data.
> - **After first power-up** (section 6): measure the current on the switched rail with the meter in series, while the station is measuring. If it is above about 20 mA, tell Eridon: the sensors should then be switched through a transistor rather than straight from the pin. I have not measured it; this is a design point to verify on the bench.

6 · Configure and flash

Open `config.h` (in the Arduino IDE it is a tab next to the sketch) and set each value:

```
#define WIFI_SSID       "Office-Wifi"            // name, exactly as shown, upper and lower case
#define WIFI_PASSWORD   "the-wifi-password"
#define API_URL         "https://your-app.up.railway.app/api/ingest"   // your domain + /api/ingest
#define API_KEY         "gk_4c078cc4_..."        // the Rehearsal key from section 3
#define SLEEP_MINUTES   1                        // 1 for the bench; set back to 10 before burn-in
#define USE_SOIL_1      1
#define USE_SOIL_2      1
#define USE_SOIL_TEMP   1
#define USE_AIR         0                        // 1 only with an SHT31
// SOIL_n_DRY / SOIL_n_WET stay at 3000 / 1300 until section 7.
```

- **Wi-Fi must be 2.4 GHz.** The ESP32 cannot join 5 GHz-only networks. A hotspot page that asks you to accept terms (a captive portal) will not work.
- **API_URL** starts with `https://` and ends with `/api/ingest`, with no spaces.
- **Set `USE_…` to 0** for any sensor you have not connected.

1. Press **Upload** (the arrow). If it stops at "Connecting……", hold the board's `BOOT` button until the upload starts, then release. If the upload fails at speed 921600, choose *Tools → Upload Speed → 115200*.
2. Open *Tools → Serial Monitor* and set the speed to **115200 baud**. Press the board's `EN` (reset) button to start a measurement cycle.

You should see the readings the station is about to send, then the platform's answer:

```
{"readings":[{"metric":"soil_moisture_1","value":31.42,"unit":"%","name":"Soil moisture 1"},
 {"metric":"soil_moisture_1_raw","value":2160.00,"unit":"raw","name":"Soil moisture 1 (raw)"},
 … soil_moisture_2, soil_moisture_2_raw, soil_temp, wifi_rssi …]}
POST 201: {"inserted":6,"alertsOpened":0,"metricsCreated":0}   (example values)
```

Then the board sleeps. After `SLEEP_MINUTES` it wakes, resets and prints the chip's boot lines before the next JSON: that is normal.

> **CHECK · THE FIRST TIME THIS FIRMWARE RUNS ON HARDWARE**
> In the platform, *Data* (filter the "Rehearsal" source) shows readings with origin *api* and fresh timestamps, and *Sites & sources* says "last data just now". **Write down anything that surprises you**, even if it seems minor.

SERIAL MONITOR SAYS	IT MEANS
`Wi-Fi not available; will retry at the next wake-up.`	Wrong name or password, 5 GHz only, or out of range. The reading is lost.
`POST 401: {"error":"Invalid API key"}`	The key is wrong, was replaced, or has a stray space or missing character.
`POST 403`	The source is deactivated on the platform.
`POST -1` (and `Upload failed`)	No connection to the server: the URL is wrong, or the Wi-Fi has no internet. See section 11 for all codes.
`POST 422` with "value must be a number"	A sensor returned something that is not a number. Send the line to Eridon.

7 · Calibrate the soil sensors

The sensor measures how much the soil changes an electric field. The board turns that raw number (0–4095) into a percentage using two reference points you measure: **DRY** (the sensor in open air) and **WET** (in water). Every sensor is a little different, so each gets its own pair. Calibrate on the **finished wiring**, powered the way it will be in the field.

1. Set `SLEEP_MINUTES` to 1 so you get a fresh reading every minute. Upload.
2. **DRY:** wipe the sensor, hold it in the air, away from your hand and from metal. Wait for 3 readings. Note `soil_moisture_1_raw` (in the Serial Monitor JSON, or on the platform under *Data*). Take the typical value.
3. **WET:** stand the sensor in a glass of water **up to the white line only**. Wait for 3 readings and note the raw value.
4. Enter both in `config.h`: `SOIL_1_DRY` and `SOIL_1_WET`. Repeat for sensor 2 with its own numbers. Upload.

Worked example (illustrative numbers)

Sensor 1 reads **2,950** in air and **1,320** in water, so `SOIL_1_DRY 2950` and `SOIL_1_WET 1320`. The firmware computes `percent = 100 × (DRY − raw) / (DRY − WET)`, limited to 0–100.
A raw reading of 2,100 gives 100 × (2950 − 2100) / (2950 − 1320) = 100 × 850 / 1630 = **52.1%**. A raw reading of 2,600 gives **21.5%**. Wetter soil gives a *lower* raw number: that is normal for this sensor.

> **CHECK**
> In air the platform shows about **0%**; in the glass about **100%**; and in between it moves smoothly as you lift the sensor out slowly. If it shows 0% in water and 100% in air, DRY and WET are swapped.

Honest limits. A percentage from a capacitive sensor is relative to *your* two reference points, not an absolute soil water content: soil type, compaction and salinity change the reading. In the field the **trend** matters more than the exact number. After installation, ask the farmer when the soil is "just irrigated" and when it "needs water", note the readings at those moments, and adjust the lower limit to match.

8 · Burn-in

Burn-in finds the faults that only appear after hours: a loose joint, a Wi-Fi dropout, a board that resets. Set `SLEEP_MINUTES` back to **10**, upload, and leave the station running **72 hours** indoors, where the farm's Wi-Fi or

router would be, with the sensors in a glass of water or in a pot of damp soil.

TEST	HOW	PASS
Availability	*Monitoring* → *From / To* over the 72 hours, "readings in period" for each metric	At least **410 of 432** (95%). Expect about 425: each cycle is a few seconds longer than 10 minutes, so about 1–2% of readings never happen.
No resets or gaps	The chart has no holes longer than one or two readings	No gap over 30 minutes
Smooth curves	*Monitoring*, each moisture metric	No spikes, no stuck values
Signal	Metric `wifi_rssi`	Always above −80 dBm
Alert chain with the real board	Set the lower limit to 28 on the Rehearsal source. Leave sensor 1 in water for 20 minutes, then lift it out into the air. Within two or three readings it falls below 28%.	An alert appears on the dashboard and **one email** arrives.

> **CHECK**
> All five pass. If one fails, find the cause (section 11) and **start the 72 hours again**. When it passes, delete the Rehearsal source, create a source for each station ("Station 1", "Station 2") with its own key, put that key in the station's `config.h` and upload.

9 · Power: mains and solar

9.1 Mains with a USB adapter (use this if you can)

1. A 5 V USB adapter of at least 1 A, plugged into the board's USB socket. Keep the adapter **indoors or under cover**; only the thin USB cable goes to the box.
2. Pass the cable through a **cable gland** in the box, with a loop hanging below the gland so rain runs off the loop and not into the box (a "drip loop").
3. Use an outdoor-rated extension if the socket is far. A farm's mains can drop out: expect some gaps and log them.

9.2 Solar (advanced; needs testing before you rely on it)

> **STATUS: REFERENCE DESIGN, NOT BUILT OR TESTED**
> Solar is the riskiest part of the pilot: November and December have short, often cloudy days, and the ESP32 board draws a few milliamps even asleep. A charger board alone **cannot** power the board: a lithium cell is 3.0–4.2 V, and the board's 5 V input needs 5 V. Build and test this on the bench for 24–48 hours **before** it goes in a field. If the farm has any mains, use 9.1.

Reference design: **6 V panel → protected TP4056 charger board → one 18650 cell → boost converter set to 5.0 V → the board's 5 V (VIN) pin and GND.**

1. Buy the **TP4056 with the battery-protection chip** (the board with separate pads for battery, output and input). Without protection a flat cell is damaged.
2. **Choose the booster by its input range.** One lithium cell is 3.0–4.2 V, so the module must accept at least 3 V (an MT3608-type module takes 2–24 V in). Some boost modules need 5 V in and will not start from one cell: read the module's specification, do not assume. I have not tested any specific module.
3. **Set the booster first.** Connect the cell only to the booster's input, turn the small screw until a multimeter on its output reads **5.0 V**, and only then connect the board. Setting it while connected can destroy the board.
4. **Never connect the USB cable and the booster at the same time**: the two 5 V sources can feed each other. Flash and configure with USB, then unplug USB before connecting the booster.
5. **Bench-test it:** run the station from a charged cell with the panel disconnected for 24–48 hours. Measure the cell voltage at the start and end. It must stay above about 3.5 V. Then place the panel in the sun and confirm that the cell charges.
6. **Place the panel** facing south, tilted about 45°, with no shade at any hour. Keep the cable short.

This is the reason the setup guide lists the solar kit as "if no mains": Eridon must design and approve it, and the burn-in must be repeated on the solar supply.

10 · Install at the farm

Bring: the station, spare board and sensors, laptop and USB cable, multimeter, cable ties, sealant, tape, a glass and a bottle of water, a phone with hotspot, and the printed farmer card.

1. **Fix the box** in the shade, above the reach of the irrigation spray, with the cable glands facing **down**. Tighten each gland until the cable cannot be pulled out, and plug any unused gland.
2. **Place the sensors in the root zone:** S1-A at 10–20 cm, S1-B at 20–40 cm (or in two spots in the plot). For hard soil make a pilot hole first with a screwdriver of similar width. Push the sensor by its **top edge only, never the electronics**, until the soil reaches the **white line**. Firm contact all round, no air gaps. Water the spot lightly so the soil settles around it.
3. **Loop each cable downward** before it enters the box, so water cannot run along it inside. Tie it to a stake so wind cannot pull the sensor.

4. **Switch on** (plug in or connect the panel). Wait 15 minutes.
5. On the platform check: *Sites & sources* shows "last data just now"; `wifi_rssi` is above **−80 dBm**; both moisture sensors show plausible percentages for soil that is damp or dry. If the signal is weak, move the router or add an extender **now**.
6. **Agree the lower limit with the farmer** (start at 25–30%, we suggest 28%), enter it on both moisture metrics, and ask when they irrigate today.
7. Show the farmer the alert email and the *Acknowledge* button, give them the farmer card, and make sure a test email reached **their own phone**.
8. **Photograph** the box, the sensor positions and the plot, and note the date: it starts the 4-week clock.

> **CHECK**
> An hour after you leave, open the platform from the office: the station is online, there are about six readings per metric, and the curves are smooth.

11 · When it does not work

SYMPTOM	DO THIS
No serial port appears	Use a data cable, not a charge-only one. Try another USB port. Install the driver for the board's USB chip (CP210x or CH340).
Upload stops at "Connecting…"	Hold the `BOOT` button while it connects. Lower the upload speed to 115200.
"Brownout detector was triggered"	The supply is too weak: use a better USB cable or adapter (≥ 1 A). On solar, the cell is flat or the booster is set wrong.
`POST -1` or `-4`	Wi-Fi joined but the server is unreachable: wrong URL, or the router has no internet (SIM out of data, captive portal). Open the URL from a phone on the same Wi-Fi.
`POST -5` or `-11`	Connection dropped or timed out: weak signal. Move closer or add an extender. If the platform is slow to wake, retry later.
`POST 401`	The key does not match. Copy it again from the password manager, or create a new one with *New API key* and paste it into `config.h`.
Moisture stuck at 0% or 100%	DRY and WET swapped, or a faulty sensor, or a loose AOUT wire. Compare the `_raw` value in air and in water: if both are the same, the sensor is dead or unplugged.
Readings jump about	A loose joint, or a sensor cable running next to a mains cable. Re-solder, re-route, and shorten the cable.
Soil temperature missing	The probe is not connected, or the 4.7 kΩ resistor is missing. The firmware skips a probe that does not answer.
Reboots in a loop ("Guru Meditation Error")	Note the full message and send it to Eridon. Try the spare board.
Works indoors, fails at the farm	It is nearly always the Wi-Fi: check `wifi_rssi` at the box's spot, not at the router.

The numbers after `POST` are the web-client's own codes when the request never completed: −1 refused, −4 not connected, −5 connection lost, −11 read timeout. Codes 200–599 are the platform's HTTP answers.

Reference · config.h and what the station sends

config.h settings

SETTING	MEANING	PILOT VALUE
`WIFI_SSID`, `WIFI_PASSWORD`	Network the station joins (2.4 GHz)	The farm's or the 4G router's
`API_URL`	Full address of the platform's ingestion endpoint	`https://…/api/ingest`
`API_KEY`	The device's key, one per station	From *Sites & sources*
`SLEEP_MINUTES`	Minutes asleep between measurements	**10** (1 only on the bench)
`USE_SOIL_1`, `USE_SOIL_2`	Moisture sensors on GPIO34 and GPIO35	1, 1
`USE_SOIL_TEMP`	DS18B20 on GPIO4	1
`USE_AIR`	SHT31 on GPIO21 and GPIO22	1 in a greenhouse, else 0
`SOIL_n_DRY`, `SOIL_n_WET`	Raw readings in air and in water, per sensor	Measured in section 7

What the station sends

METRIC KEY	UNIT	WHAT IT IS
`soil_moisture_1`, `soil_moisture_2`	%	Soil moisture, 0 = dry air, 100 = water. **These get the lower limit.**
`soil_moisture_1_raw`, `soil_moisture_2_raw`	raw	The raw sensor number, for calibration and fault-finding
`soil_temp`	°C	DS18B20 soil temperature
`air_temp`, `air_humidity`	°C, %	SHT31, only if `USE_AIR` is 1
`wifi_rssi`	dBm	Signal strength, for diagnostics. Above −80 is good.

Good to know

- **Readings are not stored on the board.** If Wi-Fi or the platform is unreachable at wake-up, that reading is lost and the station sleeps until the next one.
- The connection is encrypted (HTTPS) but the board does **not verify the server's certificate**. This is a documented pilot simplification in the firmware; the device key is the only protection. Treat the key like a password.
- The station sends no timestamp; the platform stamps each reading with its arrival time.
- Changing the firmware needs the Arduino IDE and a USB cable at the box. Plan for a laptop on site if you must change Wi-Fi details.

Commands

```
# simulated healthy reading, and simulated dry soil (use only the Rehearsal key)
python3 devices/emulator/send_test_readings.py --url https://<your-domain> --key <key>
python3 devices/emulator/send_test_readings.py --url https://<your-domain> --key <key> --dry

# platform health
curl https://<your-domain>/api/health

# compile the firmware without the IDE (optional)
arduino-cli compile --fqbn esp32:esp32:esp32 devices/esp32-soil-moisture
```